

Advanced Datatypes - Answers

Note: Code solutions are provided in the `files/svar/` directory. This file contains explanations, PBL answers, and references to the solution files.

Answer: What is an Advanced Datatype

Solution: Conceptual explanation (no code file)

Answers: An advanced datatype is a datatype that can hold more than one piece of data or has special behavior. These include:

- **Structs:** Custom datatypes that group related variables of different types under one name
- **Enums:** Named integer constants that make code more readable
- **Unions:** Special datatypes that can hold different types in the same memory location
- **Arrays:** Collections of the same datatype
- **Pointers:** Variables that store memory addresses
- **Functions:** Reusable blocks of code (can be considered advanced)

Examples already seen:

- Arrays
- Functions
- Pointers

Purpose: Advanced datatypes help organize and structure data more effectively, make code more readable and maintainable, and allow for more complex data representations.

PBL Answer: Real-world analogy for a struct: Think of a struct as a **patient record folder** that contains different sections (ID, age, height, weight). Each section holds a different type of information, but they're all grouped together in one place. Or like a **toolbox** where each compartment holds a different tool, but the whole box is one unit.

Answer: Patient Struct

Solution file: `files/svar/patient1_sol.c`

Expected Output:

```
id: 1
age: 21
heightM: 1.80
weightKg: 55.00
```

PBL Answer: Create a second patient and print both in a loop:

```
#include <stdio.h>

struct Patient {
    int id;
    int age;
    float heightM;
    float weightKg;
};

int main() {
    struct Patient tommy = {1, 21, 1.8, 55.0};
    struct Patient alice = {2, 25, 1.65, 60.0};

    struct Patient patients[] = {tommy, alice};
    int num_patients = sizeof(patients) / sizeof(patients[0]);

    for (int i = 0; i < num_patients; i++) {
        printf("Patient %d:\n", i + 1);
        printf("  id: %d\n", patients[i].id);
        printf("  age: %d\n", patients[i].age);
        printf("  height: %.2f\n", patients[i].heightM);
        printf("  weight: %.2f\n", patients[i].weightKg);
    }

    return 0;
}
```

Answer: Simple Sensor Struct

Solution file: `files/svar/sensorstruct.c`

Expected Output:

```
Sensor ID: 1
Temperature: 23.50
```

(or similar values based on what was assigned)

PBL Answer: Handle multiple sensors in an array:

```
#include <stdio.h>

struct TempSensor {
    int id;
    float temperature;
};
```

```
int main() {
    struct TempSensor sensors[3] = {
        {1, 23.5},
        {2, 24.1},
        {3, 22.8}
    };

    int num_sensors = sizeof(sensors) / sizeof(sensors[0]);

    for (int i = 0; i < num_sensors; i++) {
        printf("Sensor %d: ID=%d, Temperature=%.1f\n",
            i + 1, sensors[i].id, sensors[i].temperature);
    }

    return 0;
}
```

Answer: ESP32 Enumerate

Solution file: [files/svar/esp32_enumerate/esp32_enumerate.ino](#)

Expected Serial Monitor Output:

```
Left: 1, Right: 1
Left: 1, Right: 1
... (repeats every 100ms)
```

(Values depend on button state: 0 = pressed, 1 = not pressed with INPUT_PULLUP)

PBL Answer: Print button names instead of values:

```
#include <Arduino.h>

enum ButtonPins { Left = 0, Right = 35 };

void setup() {
    pinMode(Left, INPUT_PULLUP);
    pinMode(Right, INPUT_PULLUP);
    Serial.begin(115200);
}

void loop() {
    int leftState = digitalRead(Left);
    int rightState = digitalRead(Right);

    if (leftState == LOW) {
        Serial.println("Left pressed");
    } else {
```

```
Serial.println("Left not pressed");
}

if (rightState == LOW) {
  Serial.println("Right pressed");
} else {
  Serial.println("Right not pressed");
}

delay(100);
}
```

Answer: Patient Array

Solution file: [files/svar/patient2_sol.c](#)

Expected Output:

```
Patient 1: id=99, age=18, height=1.82, weight=88.00
Patient 2: id=93, age=35, height=1.63, weight=69.00
Patient 3: id=89, age=25, height=1.80, weight=75.00
Patient 4: id=83, age=31, height=1.51, weight=70.00
Patient 5: id=46, age=29, height=1.75, weight=63.00
Patient 6: id=16, age=56, height=1.52, weight=54.00
Patient 7: id=17, age=16, height=1.65, weight=50.00
```

PBL Answer: Calculate and print total number of patients:

```
// After defining the array:
int number_of_patients = sizeof(patients) / sizeof(patients[0]);
printf("Total number of patients: %d\n", number_of_patients);
```

Or print it before the loop:

```
printf("Patient Information for %d patients:\n", number_of_patients);
```

Answer: Patient Statistics

Solution file: [files/svar/patient3_sol.c](#)

Expected Output:

```
Patient Descriptives for 7 Patients
Age: mean = 33.00, std = 18.25
Height: mean = 1.68, std = 0.11
Weight: mean = 69.36, std = 13.14
```

(Exact values may vary based on implementation)

PBL Answer: Add min and max calculation:

```
// Add to calculations:
float min_age = ages[0], max_age = ages[0];
float min_height = heights[0], max_height = heights[0];
float min_weight = weights[0], max_weight = weights[0];

// Inside the loop:
if (ages[i] < min_age) min_age = ages[i];
if (ages[i] > max_age) max_age = ages[i];
if (heights[i] < min_height) min_height = heights[i];
if (heights[i] > max_height) max_height = heights[i];
if (weights[i] < min_weight) min_weight = weights[i];
if (weights[i] > max_weight) max_weight = weights[i];

// Print:
printf("Age: mean = %.2f, std = %.2f, min = %.2f, max = %.2f\n",
       mean_age, std_age, min_age, max_age);
```

Answer: Patient with Gender Enum

Solution file: [files/svar/patient1to4_sol.c](#)

Expected Output:

```
Male Patients:
Count: 4
Age: mean = 29.00, std = 19.72
Height: mean = 1.74, std = 0.09
Weight: mean = 74.38, std = 17.74

Female Patients:
Count: 3
Age: mean = 36.67, std = 16.33
Height: mean = 1.61, std = 0.09
Weight: mean = 62.00, std = 7.00
```

(Exact values may vary based on data)

PBL Answer: Count and print number of male and female patients:

```
// Add counters:
int male_count = 0, female_count = 0;

// In the loop where you process patients:
if (patients[i].gender == Male) male_count++;
else female_count++;

// Print counts:
printf("Number of male patients: %d\n", male_count);
printf("Number of female patients: %d\n", female_count);
```

Answer: Time Module

Solution file: `files/svar/example_timemodule.c`

Answers:

- How many bytes does `struct tm` occupy? **36 bytes** (typically 9 integers)
- When did the `time_t` counter start? **00:00 hours, Jan 1, 1970 UTC** (Unix timestamp epoch)
- What does `ctime()` do? Creates a readable string from a `time_t` value

Expected Output:

```
Size of struct tm: 36
Thu Jan  1 00:00:00 1970
```

PBL Answer: Print current time:

```
#include <stdio.h>
#include <time.h>

int main() {
    // Size of struct tm
    printf("Size of struct tm: %zu bytes\n", sizeof(struct tm));

    // Epoch time
    time_t epoch = 0;
    printf("Epoch: %s", ctime(&epoch));

    // Current time
    time_t now = time(NULL);
    printf("Current time: %s", ctime(&now));
```

```
    return 0;
}
```

Answer: Union Basics

Solution file: `files/svar/union1_sol.c` (and potentially `files/union1.c` if it exists)

Answers:

1. **Why doesn't `data.i = 10` and `data.f = -220.5` affect the output?** A union allows you to store different datatypes in the same memory location. You can only store ONE value at a time. When you assign to one member, it overwrites any previous value in the shared memory. The last assignment determines what you see. The compiler doesn't generate separate storage for each member.
2. **Relationship between `data.i` and `data.str`?** They share the same memory. `data.i` and `data.str` are different interpretations of the same bytes. If you print `data.i` as hex, you'll see the same bytes that make up the string. For example, if the string is "abc", index 3 might contain the null terminator (0x00).
3. **How much space does this union occupy? 4 bytes** (the size of the largest member, typically `int`)
 - The size of a union is determined by its largest member
 - All members share the same memory space
 - The compiler allocates enough space for the largest possible member
4. **Why doesn't `data.f` produce output?** If the union was defined with `char byte_array[4]` as a member and `data.f` was assigned but `printf` was using a format specifier for the char array, then `data.f` (4 bytes) doesn't fit into `byte_array[3]` (1 byte). The exponent in IEEE 754 float representation is stored at byte index 3, which might be 0, resulting in no output or incorrect interpretation.

PBL Answer: Demonstrate memory overlap with int/float union:

```
#include <stdio.h>

union IntOrFloat {
    int i;
    float f;
};

int main() {
    union IntOrFloat data;

    data.i = 10;
    printf("data.i = %d\n", data.i);
    printf("data.f = %f\n", data.f);
    printf("Address of data.i: %p\n", (void *)&data.i);
    printf("Address of data.f: %p\n", (void *)&data.f);
}
```

```
data.f = 3.14;
printf("\nAfter assigning float:\n");
printf("data.i = %d\n", data.i);
printf("data.f = %f\n", data.f);

return 0;
}
```

Output shows that both members share the same address, and assigning to one affects the interpretation of the other.

File References

All answer files are located in: [files/svar/](#)

- What is an Advanced Datatype: Conceptual (no code file)
- Patient Struct: [patient1_sol.c](#)
- Simple Sensor Struct: [sensorstruct.c](#)
- ESP32 Enumerate: [esp32_enumerate/esp32_enumerate.ino](#)
- Patient Array: [patient2_sol.c](#)
- Patient Statistics: [patient3_sol.c](#)
- Patient with Gender Enum: [patient1to4_sol.c](#)
- Time Module: [example_timemodule.c](#)
- Union Basics: [union1_sol.c](#)